Bangladesh Army University of Science and Technology (BAUST), Saidpur

Department of Computer Science and Engineering

Summer 2026 | CSE 2205: Database Management System

Transactions & Conflict Serializability

Lecture Note with Solved Problems and Previous Year Questions

1. What is a Transaction?

A **transaction** is a single logical unit of work that accesses and possibly updates the database. It is a sequence of operations (read, write) that is executed as **one indivisible unit**: either **all** of its operations are completed (**COMMIT**), or **none** of them take effect (**ROLLBACK**).

Example: Transferring 50 Tk from account A to account B is one transaction (PostgreSQL):

```
BEGIN;
UPDATE account SET balance = balance - 50 WHERE acc_no = 'A';
UPDATE account SET balance = balance + 50 WHERE acc_no = 'B';
COMMIT;      -- or ROLLBACK; to cancel
```

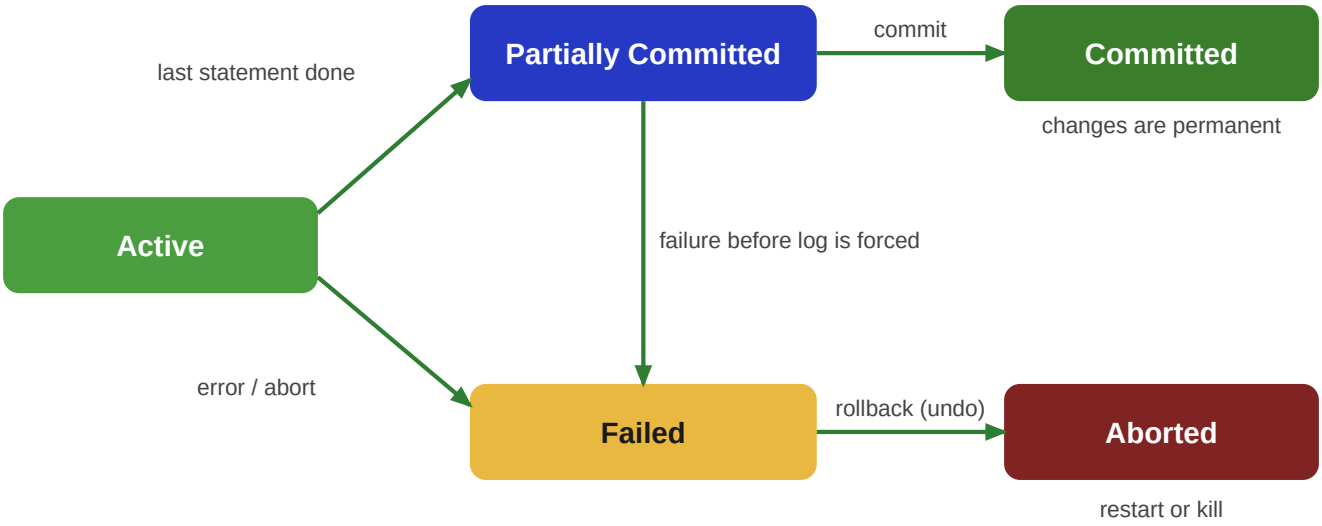
2. ACID Properties

To keep the database correct even when many transactions run at the same time or the system fails, every transaction must satisfy four properties, called **ACID**. Example used below: transfer 50 Tk from account A to account B.

Property	Explanation	Example
Atomicity	A transaction is **all or nothing**. Either all of its operations are completed, or none of them take effect. If it fails in the middle, every change it made is undone (rolled back).	If the system crashes after 50 Tk is deducted from A but before it is added to B, the deduction from A is undone.
Consistency	A transaction takes the database from one **valid (consistent) state** to another. All rules and integrity constraints remain true before and after it.	The total A + B is the same before and after the transfer; no money is created or lost.
Isolation	When transactions run **concurrently**, each one runs as if it were alone. A transaction cannot see the unfinished (uncommitted) changes of another.	Another transaction cannot read A after the deduction but before B is credited.
Durability	Once a transaction **commits**, its changes are **permanent** and survive any later failure (power cut, crash).	After the user sees "Transfer successful", the new balances stay even if the server crashes.

3. Transaction States

Every transaction moves through the following states between its first statement and its end:



State	Description
Active	Initial state; the transaction stays here while it is executing.
Partially Committed	The last statement has been executed, but changes may still be in main memory.
Committed	The transaction completed successfully; its changes are permanent.
Failed	Normal execution can no longer proceed (error, user abort, system crash).
Aborted	The transaction is rolled back and the database is restored to its state before the transaction began. The system then either **restarts** or **kills** the transaction.

4. Conflict Serializability: Solved Problems

Problem 1. Is this schedule serializable?

S: `R1(X), R2(Y), W1(X), R3(X), W2(Y), R3(Y), W3(X)`

Step	T1	T2	T3
1	R(X)		
2		R(Y)	
3	W(X)		
4			R(X)
5		W(Y)	
6			R(Y)
7			W(X)

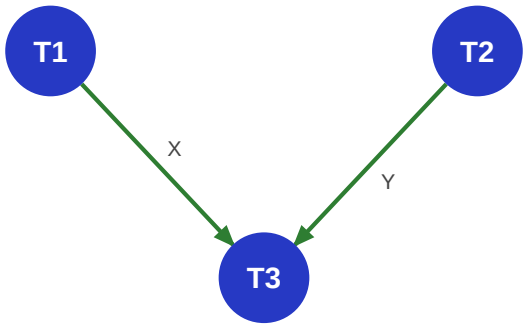
(a) List all conflicting pairs. (b) Draw the precedence graph. (c) Is S conflict serializable? If yes, give every equivalent serial order.

Solution

(a) Conflicting pairs

#	Conflicting pair	Item	Type	Edge
1	R1(X), W3(X)	X	RW	T1 → T3
2	W1(X), R3(X)	X	WR	T1 → T3
3	W1(X), W3(X)	X	WW	T1 → T3
4	W2(Y), R3(Y)	Y	WR	T2 → T3

(b) Precedence graph: T1 → T3, T2 → T3



(c) No cycle, so S is conflict serializable. T1 and T2 have no incoming edge; T3 comes last.

Answer: S is **conflict serializable**. Equivalent serial orders: **T1** → **T2** → **T3** and **T2** → **T1** → **T3**.

Problem 2. (Previous Year, Q6(a), 15 marks)

Consider the following schedule (Schedule 1) of two transactions, T1 and T2:

T1	T2
`Read(A)`	
`Write(A)`	
	`Read(A)`
	`Write(A)`
`Read(B)`	
`Write(B)`	
	`Read(B)`
	`Write(B)`

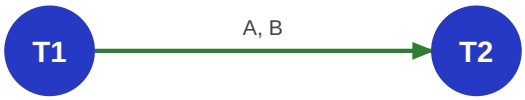
Apply conflict-serializability principles to rearrange the schedule by swapping non-conflicting operations, and determine whether these swaps can produce a conflict-equivalent serial schedule or if the schedule is not conflict-serializable.

Solution

Conflicting pairs

#	Conflicting pair	Item	Type	Edge
1	R1(A), W2(A)	A	RW	T1 → T2
2	W1(A), R2(A)	A	WR	T1 → T2
3	W1(A), W2(A)	A	WW	T1 → T2
4	R1(B), W2(B)	B	RW	T1 → T2
5	W1(B), R2(B)	B	WR	T1 → T2
6	W1(B), W2(B)	B	WW	T1 → T2

Precedence graph: only T1 → T2 (no cycle)



Swapping: R1(B) and W1(B) work on B, while R2(A) and W2(A) work on A. Operations on different items do not conflict, so T1's operations on B are swapped upward past T2's operations on A. The result is:

T1	T2
`Read(A)`	
`Write(A)`	
`Read(B)`	
`Write(B)`	
	`Read(A)`
	`Write(A)`
	`Read(B)`
	`Write(B)`

Answer: Schedule 1 is **conflict serializable**. It is conflict equivalent to the serial schedule **T1 → T2**.

Problem 3. (Previous Year, Q8(a))

Analyze the given transaction schedule (Partial Schedule 4) by constructing its precedence graph. Determine whether the schedule is conflict serializable. If it is, identify a conflict-equivalent serial schedule by performing a topological sort on the graph.

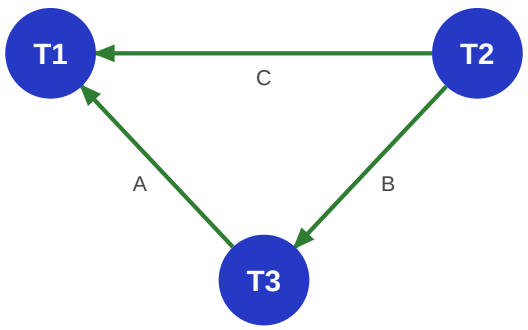
T1	T2	T3
`Read(A)`		
		`Read(B)`
		`Read(A)`
	`Read(B)`	
	`Read(C)`	
		`Write(B)`
	`Write(C)`	
`Read(C)`		
`Write(A)`		
`Write(C)`		

Solution

Conflicting pairs

#	Conflicting pair	Item	Type	Edge
1	R3(A), W1(A)	A	RW	T3 → T1
2	R2(B), W3(B)	B	RW	T2 → T3
3	R2(C), W1(C)	C	RW	T2 → T1
4	W2(C), R1(C)	C	WR	T2 → T1
5	W2(C), W1(C)	C	WW	T2 → T1

Precedence graph: T2 → T3, T3 → T1, T2 → T1 (no cycle)



Topological sort

Iteration	In-degree	Pick
1	T1 = 2, T2 = 0, T3 = 1	T2
2	T1 = 1, T3 = 0	T3
3	T1 = 0	T1

Answer: The schedule is **conflict serializable**. Equivalent serial schedule: **T2** → **T3** → **T1**.

Problem 4. (Previous Year, Q4(a), 15 marks)

Draw the precedence graph for the schedule given below. Determine whether the schedule is conflict serializable or not. If it is not conflict serializable, briefly explain the reason. If it is conflict serializable, determine the serializability order using topological sort.

Instr. No.	T1	T2	T3
1	`Read(X)`		
2	`X := X - 50`		
3			`Read(Y)`
4			`Y := X * 2`
5	`Write(X)`		
6			`Write(Y)`
7		`Read(X)`	
8		`X := X + 30`	
9		`Write(X)`	
10	`Read(Y)`		
11	`Y := Y + 10`		
12	`Write(Y)`		
13			`Read(Y)`
14		`Read(Y)`	

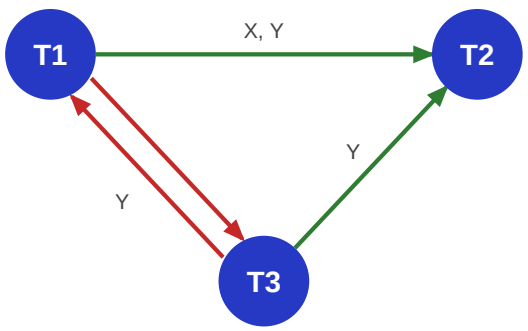
Solution

Only Read/Write operations are considered; computations like `X := X - 50` do not cause conflicts.

Conflicting pairs

#	Conflicting pair	Item	Type	Edge
1	R1(X), W2(X)	X	RW	T1 → T2
2	W1(X), R2(X)	X	WR	T1 → T2
3	W1(X), W2(X)	X	WW	T1 → T2
4	R3(Y), W1(Y)	Y	RW	T3 → T1
5	W3(Y), R1(Y)	Y	WR	T3 → T1
6	W3(Y), W1(Y)	Y	WW	T3 → T1
7	W3(Y), R2(Y)	Y	WR	T3 → T2
8	W1(Y), R3(Y)	Y	WR	**T1 → T3**
9	W1(Y), R2(Y)	Y	WR	T1 → T2

Precedence graph: T1 → T2, T3 → T1, T1 → T3, T3 → T2



Reason: T3 writes Y before T1 reads and writes Y (needs T3 before T1), but T3 reads Y again after T1 writes it (needs T1 before T3). This forms the cycle **T1 → T3 → T1** (red).

Answer: The graph has a cycle, so the schedule is **NOT conflict serializable**, and no serial order exists.